

Pipeline de Donnees

Systeme de Recommandation Zero-Click

Nettoyage | Transformation | Feature Engineering

Auteur :	Omar Chekroun
Projet :	PFA 2026 - Recommandation ferroviaire
Dataset initial :	946 155 reservations ONCF
Dataset final :	491 680 lignes 69 449 clients 1 067 liaisons
Features construites :	26 colonnes (7 categoriel + 15 numerique + 4 meta)

1.	Table des matieres	3
2.	1. Sources de donnees brutes	3
2.	2. Vue d'ensemble du pipeline	4
3.	3. Etape 1 - Nettoyage des donnees (cleaning.py)	5
3.1	3.1 Validation des colonnes obligatoires	5
3.2	3.2 Conversion des types	5
3.3	3.3 Colonnes supprimees (constantes)	6
3.4	3.4 Jointure avec Liaison.csv	7
3.5	3.5 Suppression des valeurs manquantes	7
3.6	3.6 Detection et suppression des annulations	8
3.7	3.7 Suppression des clients cold-start	10
3.8	3.8 Encodage cyclique du temps	11
4.	4. Etape 2 - Feature Engineering (features.py)	12
4.1	4.1 Les 7 features categorielles	13
4.2	4.2 Les features numeriques	14
4.3	4.3 La feature cle : user_top_liaison_share	15

4.4	Inference en production : compute_inference_row	16
5.	Schema final des colonnes	17
6.	Bilan et chiffres cles	18

1. Sources de donnees brutes

Le systeme de recommandation part de deux fichiers CSV bruts fournis par l'ONCF. Ces fichiers representent les reservations historiques des clients et la table de reference des liaisons ferroviaires.

Fichier 1 - oncf_data.csv (reservations)

946 155 lignes. Chaque ligne represente une reservation de billet par un client.

Colonne	Type brut	Description
CodeClient	str/int	Identifiant unique du client (cle de lookup - jamais
LiaisonVoyageurSegmentIdSTG	str/float	Code de la liaison (devient LiaisonId apres
DateHeureDepartVoyageSegment	str/datetime	Date et heure de depart du voyage - colonne
DatePaiement	str/datetime	Date de paiement de la reservation
TypeParcoursId	str/int	Type de trajet : aller simple, aller-retour, etc.
ClassificationId	str/int	Classification de la reservation
ClassePhysiqueId	str/int	Classe physique du train (1ere, 2eme...)
NiveauPrixId	str/int	Niveau de prix / tarif applique
TrainAutocarId	str/int	Indicateur train ou autocar
CarteClientId	str/int	Type de carte de fidelite du client
PrixParLiaison	str/float	Prix paye pour cette liaison (peut etre 0 = 100%
NbrVoySegment	str/float	Nombre de segments - clef pour detecter les
DelaiAnticipation	str/float	Nombre de jours entre achat et depart
TrajetAllerRetour	str	Identifiant du trajet aller-retour (optionnel)
AcheteurId	str	Identifiant de l'acheteur (? CodeClient si achat pour
PosteVenteld	float	ID du poste de vente - SUPPRIME (constante)
TypeOperationVenteApresVenteld	float	Type operation apres-vente - SUPPRIME
PrixServices	float	Prix des services - SUPPRIME (constante)

Fichier 2 - Liaison.csv (referentiel routes)

Table de correspondance entre les identifiants de liaison et les noms des gares.

Colonne	Description
LiaisonId	Identifiant unique de la liaison ferroviaire
DesignationFrGareDepart	Nom complet de la gare de depart en francais
DesignationFrGareArrivee	Nom complet de la gare d'arrivee en francais

Jointure entre les deux fichiers

oncf_data.csv contient LiaisonVoyageurSegmentIdSTG (code brut). Apres normalisation (suppression du '.0' final), ce code est joint avec LiaisonId de Liaison.csv pour recuperer les noms des gares. Taux de correspondance : 99.92% (1 seule ligne sans match).

2. Vue d'ensemble du pipeline de donnees

Le pipeline transforme les donnees brutes en features pret pour l'entrainement en deux grandes etapes executees par deux scripts independants.

1

Nettoyage (script 01_make_dataset.py)

cleaning.py -> make_clean_dataset()

IN: oncf_data.csv + Liaison.csv

OUT: oncf_clean.parquet (491 680 lignes)

2

Feature Engineering (script 02_build_features.py)

features.py -> build_training_rows()

IN: oncf_clean.parquet (491 680 lignes)

OUT: features.parquet (491 680 x 26 colonnes)

3

Entrainement XGBoost (script 03_train_ranker.py)

training.py -> train_xgb_multiclass()

IN: features.parquet

OUT: xgb_ranker.json + label_encoder.joblib

Bilan numerique du nettoyage

Etape	Lignes supprimees	% du total
Lignes initiales (oncf_data.csv)	946 155	100.0%
- Pas de correspondance Liaison.csv	1	< 0.01%
- Valeurs trop manquantes	0	0.00%
- Annulations + reservations annulees	50 859	5.37%
- Champs essentiels manquants	0	0.00%
- Doublons exacts	0	0.00%
- Clients cold-start (< 3 voyages)	403 615	42.66%
= TOTAL SUPPRIME	454 475	48.03%
= DATASET FINAL (oncf_clean.parquet)	491 680	51.97%

3. Etape 1 - Nettoyage des données (cleaning.py)

La fonction `make_clean_dataset(oncf_df, liaison_df)` est le cœur du pipeline de nettoyage. Elle prend les deux dataframes bruts en entrée et retourne : (1) le dataframe nettoyé, (2) un rapport de nettoyage JSON, (3) une table de provenance qui enregistre la décision prise pour chaque ligne originale.

3.1 Validation des colonnes obligatoires

Avant toute transformation, le code vérifie que les colonnes requises sont présentes. Si une seule colonne obligatoire est absente, une exception est levée immédiatement.

```
REQUIRED_ONCF_COLS = [  
    'TrajetAllerRetour', 'TypeParcoursId', 'CodeClient', 'ClassificationId',  
    'ClassePhysiqueId', 'NiveauPrixId', 'TrainAutocarId', 'LiaisonVoyageurSegmentIdSTG',  
    'CarteClientId', 'PrixParLiaison', 'NbrVoySegment', 'DatePaiement',  
    'DateHeureDepartVoyageSegment', 'DelaiAnticipation'  
]  
missing = [c for c in REQUIRED_ONCF_COLS if c not in oncf_df.columns]  
if missing:  
    raise ValueError(f'Missing columns: {missing}')
```

Pourquoi ?

Le système sera déployé en production et recevra de nouveaux exports CSV périodiquement. La validation stricte à l'entrée garantit qu'une erreur de schéma (colonne renommée, manquante) est détectée immédiatement plutôt que de produire des données silencieusement corrompues.

3.2 Conversion des types

Les CSV sont chargés avec des types génériques (str/object). Le code convertit explicitement chaque colonne vers son type correct.

```
# Dates : deux formats tentés (ISO puis DD/MM/YYYY)  
df['DatePaiement'] = pd.to_datetime(series, errors='coerce')  
# Si > 20% échouent, retente avec dayfirst=True  
  
# IDs et prix : conversion numérique, erreurs -> NaN  
for col in numeric_cols:  
    df[col] = pd.to_numeric(df[col], errors='coerce')  
  
# AcheteurId : intentionnellement EXCLU de la conversion numérique  
# C'est un identifiant de personne, pas une quantité.
```

- `DateHeureDepartVoyageSegment` : `datetime64` - colonne temporelle principale pour le tri et le split
- `TypeParcoursId`, `ClassePhysiqueId`, etc. : convertis en `float64` - indispensable pour la détection des valeurs négatives (annulations)
- `AcheteurId` : reste str - compare avec `CodeClient` pour `is_self_purchase`

- PrixServices, PosteVenteld, TypeOperationId : convertis mais supprimes ensuite (constantes)

3.3 Colonnes supprimees - Colonnes constantes

Une colonne 'constante' est une colonne dont tous les valeurs non-nulles sont identiques. Elle n'apporte aucun signal discriminant pour le modele (variance = 0).

```
def _drop_constant_columns(df, *, ignore):
    constant_cols = []
    for column in df.columns:
        if column in ignore: continue
        non_null = df[column].dropna()
        if non_null.empty or non_null.nunique() <= 1:
            constant_cols.append(column)
    return df.drop(columns=constant_cols), constant_cols
```

3 colonnes ont ete supprimees dans notre dataset :

Colonne supprimee	Valeur unique observee	Raison de suppression
PosteVenteld	Constante	Une seule valeur - zero information pour le
TypeOperationVenteApresVenteld	Constante	Idem - aucune variabilite
PrixServices	Constante	Idem - aucune variabilite

Colonnes protegees (non testees pour la constance)

CodeClient, LiaisonVoyageurSegmentIdSTG, TrajetAllerRetour, AcheteurId, DatePaiement, DateHeureDepartVoyageSegment. Ces colonnes sont des cles ou des identifiants qui pourraient techniquement etre constantes sans que cela soit une erreur - elles sont donc ignorees dans le test de constance.

3.4 Normalisation de LiaisonId et jointure

La colonne LiaisonVoyageurSegmentIdSTG contient des codes comme '123.0', '456.0'. Ces suffixes '.0' apparaissent quand pandas lit une colonne d'entiers qui contient des NaN (converti en float). La normalisation supprime ce suffixe avant la jointure.

```
# Normalisation : '123.0' -> '123'
df['LiaisonId'] = series.astype(str).str.replace(r'\.0$', '', regex=True).str.strip()

# Jointure LEFT pour conserver toutes les reservations
# et ajouter DesignationFrGareDepart + DesignationFrGareArrive
out = df.merge(liaison, on='LiaisonId', how='left')

# Lignes sans correspondance -> supprimees (1 ligne dans notre dataset)
join_missing = out[['DesignationFrGareDepart', 'DesignationFrGareArrive']].isna().any(axis=1)
out = out.loc[~join_missing]
```

- Taux de correspondance : 99.92% - 1 290 LiaisonId distincts sur 1 291 trouvent leur match
- 1 seule ligne supprimee faute de correspondance dans notre dataset
- DesignationFrGareDepart / DesignationFrGareArrive : utilisees uniquement pour l'affichage UI, pas

comme features

3.5 Suppression des lignes avec trop de valeurs manquantes

Une ligne est supprimée si elle a PLUS DE 1 valeur manquante parmi toutes ses colonnes (en excluant les colonnes de noms de gares qui peuvent légitimement être absentes). Dans notre dataset : 0 ligne supprimée par cette règle.

Champs essentiels

CodeClient, LiaisonId, DateHeureDepartVoyageSegment sont vérifiées séparément : si l'un de ces trois champs est manquant, la ligne est supprimée indépendamment du nombre total de valeurs manquantes. 0 cas dans notre dataset.

3.6 Detection et suppression des annulations

C'est l'étape la plus complexe du nettoyage. 50 859 lignes sont supprimées. Une annulation dans le système ONCF est encodée comme une nouvelle ligne dans le CSV, pas comme la modification d'une ligne existante. Il faut donc la détecter et supprimer AUSSI la réservation d'origine qu'elle annule.

Comment reconnaître une ligne d'annulation ?

Indicateur	Condition	Explication métier
NbrVoySegment <= 0	nb_segments = 0	Zero passager = voyage annulé. Note : 0
PrixParLiaison < 0	prix < 0 (négatif)	Montant négatif = remboursement /
Autres IDs < 0	TypeParcoursId < 0, etc.	Identifiants négatifs = lignes d'inversion comptable générées automatiquement.

```
is_cancellation = pd.Series(False, index=ordered.index)

# Critere 1 : zero segment = annulation
is_cancellation |= ordered['NbrVoySegment'].le(0)

# Critere 2 : prix negatif = remboursement
# (IMPORTANT : .lt(0) et non .le(0) car PrixParLiaison=0 est valide !)
is_cancellation |= ordered['PrixParLiaison'].lt(0)

# Critere 3 : IDs categoriel negatifs = reversal comptable
is_cancellation |= ordered[cancel_strict_neg_cols].lt(0).any(axis=1)
```

Comment identifier la réservation annulée ?

Une annulation void la réservation qui la PRÉCEDE dans l'historique du même client. Le code trie d'abord par (CodeClient, DateHeureDepartVoyageSegment), puis pour chaque ligne d'annulation, récupère la ligne précédente via un groupby + shift(1).

```
# Tri par client et date
ordered = out.sort_values(['CodeClient', 'DateHeureDepartVoyageSegment'])

# Pour chaque ligne, l'_orig_row_id de la ligne précédente du même client
group_prev_orig = ordered.groupby(['CodeClient', 'TrajetAllerRetour'])['_orig_row_id'].shift(1)

# IDs des lignes d'annulation
neg_orig_ids = set(ordered.loc[is_cancellation, '_orig_row_id'])

# IDs des réservations qui précédaient ces annulations
prev_orig_ids = set(group_prev_orig.loc[is_cancellation].dropna().astype(int))

# Supprimer les deux : l'annulation ET la réservation d'origine
indices_to_drop = neg_orig_ids | prev_orig_ids
```

Exemple concret pour un client :

Client 12345 - historique trie par date :

Ligne A (2025-01-10) : Casablanca -> Rabat | NbrVoy=1, Prix=120 <- RESERVATION ORIGINALE

Ligne B (2025-01-12) : Casablanca -> Rabat | NbrVoy=0, Prix=-120 <- ANNULATION (void Ligne A

=> Lignes A et B supprimees toutes les deux

Chiffres clés - annulations

32 010 lignes d'annulation detectees

+ 18 849 reservations d'origine supprimees (la reservation qui precedait chaque annulation)

= 50 859 lignes supprimees au total (5.37% du dataset initial)

3.7 Suppression des clients cold-start

C'est la plus grande suppression : 403 615 lignes, 304 595 clients. Un client 'cold-start' est un client avec moins de 3 réservations au total dans le dataset.

La règle : MIN_TRIPS_FOR_TRAINING = 3

```
MIN_TRIPS_FOR_TRAINING = 3

# Compte le nombre de voyages par client
trip_counts = out.groupby('CodeClient').size()

# Identifie les clients sous le seuil
cold_start_clients = trip_counts[trip_counts < MIN_TRIPS_FOR_TRAINING].index

# Supprime TOUTES les lignes de ces clients
cold_start_mask = out['CodeClient'].isin(cold_start_clients)
out = out[~cold_start_mask]
```

Pourquoi supprimer ces clients du training ?

Trois raisons complémentaires justifient cette décision :

1. Pas de signal apprennable

Un client avec 1 ou 2 voyages n'a pas d'historique suffisant pour que le modèle apprenne son comportement. Inclure ces lignes ajoute du bruit sans signal utile. Le modèle ne peut pas généraliser depuis 1 exemple.

2. Coherence train / production

En production, ces mêmes clients (< 3 voyages) ne passent PAS par le modèle XGBoost. Ils reçoivent une recommandation par popularité globale (cold-start path). Les inclure dans le training créerait une incohérence : entraînerait sur des profils qu'on ne modélise pas en prod.

3. Alignement métier

L'objectif est de prédire le comportement des voyageurs réguliers de l'ONCF. Un client avec 1 voyage pourrait être occasionnel ou un touriste - pas la cible principale.

Que se passe-t-il pour ces clients en production ?

En production, si un client a < 3 voyages dans l'historique, le Recommender applique un fallback en cascade :

```
if len(history) < 3:
    # Essai 1 : co-occurrence (cold_start.joblib)
    result = cold_start_rec.recommend() # mode: 'cold_start_cf'
    if not result:
        # Essai 2 : popularité globale (popularity.joblib)
        return _fallback(k) # mode: 'popularity'
```

- mode: 'cold_start_cf' - recommandation par co-occurrence (qui prend quelles routes en même temps)

- mode: 'popularity' - top-k global des liaisons les plus reservees
- mode: 'cold_start' - liste vide (uniquement si popularity.joblib est absent)

Impact sur la distribution des clients

Avant nettoyage : ~374 044 clients distincts

Après suppression cold-start : 69 449 clients actifs (avec ≥ 3 voyages)

Cela signifie que 304 595 clients (81.4% des clients) ont moins de 3 voyages. Ce chiffre est typique des systemes de reservation ferroviaire : la majorite des clients voyagent rarement, et seule une minorite est reguliere.

3.8 Encodage cyclique du temps

Après le nettoyage, le code extrait trois features temporelles brutes depuis DateHeureDepartVoyageSegment, puis les encode de manière cyclique.

Les trois features temporelles brutes

Feature	Calcul	Plage de valeurs
depart_hour	dt.hour	0 a 23
depart_dow	dt.dayofweek	0 (lundi) a 6 (dimanche)
depart_month	dt.month	1 a 12

Pourquoi encoder cycliquement ?

Si on donne les valeurs brutes au modèle, il voit que l'heure 23 et l'heure 0 sont très éloignées ($|23 - 0| = 23$). Mais métrologiquement, 23h et 0h sont adjacentes (5 minutes d'écart possible). L'encodage cyclique résout ce problème en projetant chaque valeur sur un cercle via sinus et cosinus.

```
# Formule generale pour une periode P :
sin_feature = sin(2 * pi * valeur / P)
cos_feature = cos(2 * pi * valeur / P)

# Heure (P=24) :
depart_hour_sin = sin(2 * pi * depart_hour / 24)
depart_hour_cos = cos(2 * pi * depart_hour / 24)

# Jour semaine (P=7) :
depart_dow_sin = sin(2 * pi * depart_dow / 7)
depart_dow_cos = cos(2 * pi * depart_dow / 7)

# Mois (P=12) :
depart_month_sin = sin(2 * pi * depart_month / 12)
depart_month_cos = cos(2 * pi * depart_month / 12)
```

Propriété clé : continuité aux extrêmes

Avec l'encodage cyclique (sin, cos), la distance entre heure 23 et heure 0 est petite - exactement comme la distance entre heure 1 et heure 2. Le modèle peut donc détecter que 'voyage de nuit' (22h-1h) est une catégorie cohérente sans qu'une frontière artificielle brise le signal. On utilise TOUJOURS les deux (sin ET cos) car sin seul est ambigu : $\sin(30 \text{ degrés}) = \sin(150 \text{ degrés})$, donc deux heures différentes auraient le même sin. Avec sin + cos ensemble, chaque heure a une paire (sin, cos) unique.

6 colonnes ajoutées au total par l'encodage cyclique

depart_hour_sin, depart_hour_cos (heure de départ)
 depart_dow_sin, depart_dow_cos (jour de la semaine)
 depart_month_sin, depart_month_cos (mois de l'année)

Ces 6 colonnes s'ajoutent aux 3 colonnes brutes (depart_hour, depart_dow, depart_month) qui sont également

conservees car XGBoost peut exploiter les deux representations.

4. Etape 2 - Feature Engineering (features.py)

La fonction `build_training_rows(clean_df)` transforme le dataset nettoye en table d'entrainement. Elle produit une ligne par voyage historique, ou chaque ligne represente 'ce que le modele savait au moment de ce voyage' pour predire la liaison.

Entree : `oncf_clean.parquet` (491 680 lignes)

Sortie : `features.parquet` (491 680 lignes x 26 colonnes)

```
def build_training_rows(clean_df):
    df = clean_df.copy()
    df = df.sort_values(['CodeClient', 'DateHeureDepartVoyageSegment'])

    # Features calculees par groupe (historique de chaque client)
    df['user_trip_index'] = df.groupby('CodeClient').cumcount()
    df['prev_liaison'] = df.groupby('CodeClient')['LiaisonId'].shift(1)
    df['days_since_prev'] = ... # difference de dates
    df['user_top_liaison_share'] = ... # rolling fraction (voir section 4.3)
    df['is_self_purchase'] = (df['AcheteurId'] == df['CodeClient']).astype(int)

    # Conversion des categories en str pour OrdinalEncoder
    for c in cat_cols:
        df[c] = df[c].astype('Int64').astype(str)

    return df[out_cols]
```

4.1 Les 7 features categorielles

Ces features sont passees dans un `OrdinalEncoder` (pas `OneHotEncoder`) pour deux raisons : (1) `XGBoost` gere bien les entiers ordinaux, (2) `prev_liaison` a 1 011 valeurs uniques - OHE creerait 1 011 colonnes supplementaires.

Feature	Source	Description et signal
TypeParcoursId	brut	Type de trajet : aller simple, aller-retour, etc. Influence la
ClassificationId	brut	Classification de la reservation (abonnement, billet
ClassePhysiqueId	brut	Classe physique : 1ere ou 2eme classe. Corre le au
NiveauPrixId	brut	Niveau de tarif applique. Capte la sensibilite au prix.
TrainAutocarId	brut	Train ou autocar. Certaines liaisons n'existent qu'en
CarteClientId	brut	Type de carte fidelite. Proxy du profil voyageur.
prev_liaison	calcule	Liaison precedente du client. Feature la plus predictive : les voyageurs repetent souvent leurs routes.

prev_liaison - la feature categorielle la plus importante

`prev_liaison` = la `LiaisonId` du voyage precedent du client (calculee par `groupby + shift(1)`).

NaN pour le premier voyage d'un client -> converti en la chaine 'nan' pour `OrdinalEncoder`.

Signal : un voyageur qui a pris Casablanca->Rabat la derniere fois a une forte probabilite de reprendre la meme

route. La baseline 'prev_liaison' (toujours recommander la dernière route) atteint $HR@1 = 0.26$ - le modèle monte à 0.76 en intégrant le contexte complet.

4.2 Les features numeriques

Feature	Calcul	Description
CodeClient	brut (int64)	ID client - CLE DE LOOKUP UNIQUEMENT.
PrixParLiaison	brut (float64)	Prix paye. Capte les habitudes de budget.
NbrVoySegment	brut (float64)	Nombre de segments du voyage.
DelaiAnticipation	brut (float64)	Jours entre achat et depart. Planifie=grand,
user_trip_index	cumcount()	Numero du voyage dans l'historique client.
days_since_prev	diff. de dates	Jours depuis le voyage precedent. Capte le
user_top_liaison_share	rolling frac.	Part de la liaison la plus frequente dans
depart_hour	dt.hour	Heure brute (0-23). Conservee en plus du
depart_dow	dt.dayofweek	Jour semaine brut (0=Lundi). Conserve en
depart_month	dt.month	Mois brut (1-12). Conserve en plus du
depart_hour_sin/cos	sin/cos cyclique	Encodage cyclique de l'heure (periode=24).
depart_dow_sin/cos	sin/cos cyclique	Encodage cyclique du jour (periode=7).
depart_month_sin/cos	sin/cos cyclique	Encodage cyclique du mois (periode=12).
is_self_purchase	AchteurId==CodeClient	1 si achat pour soi-meme, 0 si achat pour quelqu'un d'autre. Proxy du type de deplacement.

4.3 La feature cle : user_top_liaison_share

Cette feature a été ajoutée en Sprint 2 et a produit la plus grande amélioration des métriques (+2.33pp sur HR@1, de 0.7395 à 0.7628). Elle mesure le degré de 'fidélité' d'un utilisateur à une route dominante.

Definition

Pour un utilisateur à la position i dans son historique trié par date : $\text{user_top_liaison_share} = (\text{nombre de fois où la liaison la plus fréquente apparaît dans les } i-1 \text{ voyages précédents}) / (i-1)$.

Exemples :

- 0.9 = 90% de mes voyages passés sont sur la même route (très fidèle)
- 0.3 = je prends plusieurs routes différentes (voyageur varie)
- NaN = premier voyage (aucun historique)

Implementation : calcul strictement sur le passé (sans data leakage)

```
def _rolling_top_share(series):
    counts = Counter()
    out = np.empty(len(series), dtype=float)
    out[0] = np.nan # NaN pour le 1er voyage : pas d'historique

    for i, value in enumerate(series.tolist()):
        if i > 0:
            # Fraction de la liaison la plus fréquente
            # dans les i observations PRECEDENTES
            top_n = counts.most_common(1)[0][1]
            out[i] = top_n / float(i)
            counts[str(value)] += 1 # mise à jour APRES le calcul
    return pd.Series(out, index=series.index)

# Appliquée par groupe client
df['user_top_liaison_share'] = (
    df.groupby('CodeClient')['LiaisonId']
    .apply(_rolling_top_share)
)
```

Pourquoi 'rolling' et pas global ?

Si on calculait la part globale (incluant les voyages futurs), la ligne du voyage N contiendrait des informations sur les voyages $N+1$, $N+2$... qui n'existaient pas encore au moment du voyage N . C'est du DATA LEAKAGE - le modèle verrait le futur pendant l'entraînement et serait artificiellement bon. La version rolling calcule uniquement depuis les observations passées : la ligne du voyage i n'utilise que les voyages 0 à $i-1$.

Impact sur les métriques (Sprint 2)

Avant user_top_liaison_share : HR@1 = 0.7395 (Sprint 1)

Après user_top_liaison_share : HR@1 = 0.7628 (Sprint 2) -> +2.33 points de pourcentage

Le modele capture maintenant le profil de fidelite de chaque utilisateur, ce qui lui permet de distinguer les voyageurs 'monotones' (recommander toujours la meme route) des voyageurs 'varies' (explorer plus de candidats).

4.4 Inference en production : compute_inference_row()

build_training_rows() produit les donnees d'entrainement : une ligne par voyage historique. Mais en production, quand un client ouvre l'application, on ne connait pas encore son prochain voyage. On doit construire UNE seule ligne de features qui represente 'a quoi ressemble ce client maintenant', pour predire ce qu'il va reserver.

C'est le role de compute_inference_row(history_df, asof=now).

Differences cles avec build_training_rows

Aspect	build_training_rows (training)	compute_inference_row (prod)
Nombre de lignes	1 ligne par voyage historique	1 seule ligne (la prediction)
Date de reference	DateHeureDepartVoyageSegment	asof = maintenant (quand l'app s'ouvre)
prev_liaison	shift(1) = liaison du voyage	last['LiaisonId'] = derniere liaison
user_trip_index	cumcount() = index cumule	len(history) = total voyages connus
days_since_prev	diff entre voyage N et N-1	(maintenant - derniere date) en jours
user_top_liaison_share	fraction sur i-1 obs. passees	fraction sur TOUT l'historique connu
Colonnes prix/classe	valeurs de la ligne courante	proxy = valeurs du DERNIER voyage
LiaisonId (cible)	vraie valeur (etiquette d'entrainement)	'__unknown__' - supprimee avant predict

```
def compute_inference_row(history_df, asof=None):
    if asof is None:
        asof = pd.Timestamp(datetime.now())

    sorted_hist = history_df.sort_values('DateHeureDepartVoyageSegment')
    last = sorted_hist.iloc[-1] # dernier voyage connu

    user_trip_index = float(len(sorted_hist))
    days_since_prev = (asof - last['DateHeureDepartVoyageSegment']).total_seconds() / 86400

    # user_top_liaison_share sur tout l'historique
    liaison_counts = sorted_hist['LiaisonId'].value_counts()
    top_share = liaison_counts.iloc[0] / len(sorted_hist)

    # Features temporelles basees sur 'maintenant'
    row['depart_hour'] = asof.hour
    row['depart_hour_sin'] = sin(2*pi*asof.hour/24)
    # ... etc

    return pd.DataFrame([row]) # 1 ligne
```

Pourquoi proxy les features prix/classe avec le dernier voyage ?

En production, on ne connait pas encore les details du PROCHAIN voyage (prix, classe...). On utilise le dernier voyage connu comme approximation : l'hypothese est que le prochain voyage ressemblera au dernier en termes de classe et de prix. C'est un proxy imparfait mais c'est la meilleure approximation disponible sans information sur le futur.

5. Schema final des colonnes (features.parquet - 26 colonnes)

Voici la liste complete des 26 colonnes du fichier features.parquet, avec leur role dans le pipeline.

Colonne	Type	Role	Description
DateHeureDepartVoyageSegment	datetime64	META	Date de depart - pour temporal split,
LiaisonId	str	CIBLE	La liaison a predire (1 011 classes)
CodeClient	float64	META	ID client - cle de lookup, jamais
TypeParcoursId	str	CAT	Type de trajet (OrdinalEncoder)
ClassificationId	str	CAT	Classification reservation
ClassePhysiqueId	str	CAT	Classe physique (OrdinalEncoder)
NiveauPrixId	str	CAT	Niveau de prix (OrdinalEncoder)
TrainAutocarId	str	CAT	Train ou autocar (OrdinalEncoder)
CarteClientId	str	CAT	Carte de fidelite (OrdinalEncoder)
prev_liaison	str	CAT	Liaison precedente, 'nan' si 1er voyage
PrixParLiaison	float64	NUM	Prix paye (nullable)
NbrVoySegment	float64	NUM	Nombre de segments
DelaiAnticipation	float64	NUM	Jours anticipes avant depart
user_trip_index	int64	NUM	Index cumule du voyage du client
days_since_prev	float64	NUM	Jours depuis le voyage precedent
user_top_liaison_share	float64	NUM	Part liaison dominante (rolling, pas de
depart_hour	int32	NUM	Heure de depart brute (0-23)
depart_dow	int32	NUM	Jour semaine brut (0=Lundi)
depart_month	int32	NUM	Mois brut (1-12)
depart_hour_sin	float64	NUM	$\sin(2 \cdot \pi \cdot \text{heure} / 24)$
depart_hour_cos	float64	NUM	$\cos(2 \cdot \pi \cdot \text{heure} / 24)$
depart_dow_sin	float64	NUM	$\sin(2 \cdot \pi \cdot \text{dow} / 7)$
depart_dow_cos	float64	NUM	$\cos(2 \cdot \pi \cdot \text{dow} / 7)$
depart_month_sin	float64	NUM	$\sin(2 \cdot \pi \cdot \text{mois} / 12)$
depart_month_cos	float64	NUM	$\cos(2 \cdot \pi \cdot \text{mois} / 12)$
is_self_purchase	int64	NUM	1 si acheteur = client, 0 sinon

META

CIBLE

CAT = categorielle

NUM = numerique

6. Bilan et chiffres clés

Chiffres clés du pipeline

Metrique	Valeur	Contexte
Lignes initiales	946 155	
Lignes apres annulations	895 295	-50 859 (-5.4%)
Lignes apres cold-start	491 680	-403 615 (-45.1%)
Clients actifs (>= 3 voyages)	69 449	18.6% des clients initiaux
Liaisons distinctes	1 067	routes ferroviaires differentes
Colonnes features finales	26	7 cat + 15 num + 4 meta
Colonnes supprimees (constantes)	3	PosteVenteld, TypeOpVAV, PrixServices
Taux de jointure Liaison.csv	99.92%	1 290 / 1 291 LiaisonIds matchent

Decisions d'architecture importantes

OrdinalEncoder plutot que OneHotEncoder

prev_liaison a 1 011 valeurs uniques. OneHotEncoder creerait 1 011 colonnes = 5 000+ colonnes au total. Impossible a gerer par XGBoost dans ce contexte. OrdinalEncoder garde tout en 1 colonne avec des entiers (0, 1, 2...).

Encodage cyclique plutot que brut pour le temps

Les valeurs brutes (0-23 pour l'heure) creent une discontinuite artificielle aux extremes. L'encodage sin/cos preserve la continuite : heure 23 et heure 0 sont proches.

Rolling share plutot que global share pour user_top_liaison_share

Calculer la fraction sur l'historique global inclurait les voyages futurs (data leakage). La version rolling n'utilise que les observations strictement anterieures a chaque ligne.

Suppression des clients cold-start (< 3 voyages)

Ces clients representent 81.4% des clients mais n'ont pas assez de signal pour etre appris par le modele. Ils sont geres separement en production par un fallback cascade.

CodeClient jamais feature du modele

Loi 09-08 (equivalent RGPD marocain) / CNDP. CodeClient est utilise uniquement comme cle de lookup pour recuperer l'historique. Il n'est jamais passe au modele XGBoost, jamais logue dans les requetes API.

Proxy des features de voyage futures par le dernier voyage connu

En production, les details du prochain voyage (prix, classe...) sont inconnus. On utilise le dernier voyage comme approximation, ce qui est justifie par la regularite des comportements des voyageurs ferroviaires.

Recapitulatif visuel du pipeline complet

DONNEES BRUTES

oncf_data.csv	Liaison.csv
946 155 reservations	1 290 liaisons + noms de gares



- SCRIPT 01 - Data Cleaning**
- 1. Identification et suppression des doublons
 - 2. Conversion types (dates, numeriques)
 - 3. Suppression 3 colonnes constantes (PosteVentId, TypeOpVAV, PrixServices)
 - 4. Normalisation LiaisonId + LEFT JOIN Liaison.csv
 - 5. Suppression lignes sans match (-1 ligne)
 - 6. Detection annulations (NbrVoySegment<=0, Prix<0, IDs<0)
+ Suppression annulation ET reservation d'origine (-50 859 lignes)
 - 7. Suppression champs essentiels manquants (-0 lignes)
 - 8. Suppression doublons exacts (-0 lignes)
 - 9. Ajout features temporelles (depart_hour, dow, month + sin/cos x3)
 - 10. Suppression clients < 3 voyages (cold-start) (-403 615 lignes, -304 595 clients)

RESULTAT : oncf_clean.parquet : 491 680 lignes | 69 449 clients | 1 067 liaisons



- SCRIPT 02 - Client Data Engineering**
- 1. Calcul client_trip_index : groupby cumcount
 - 2. Calcul user_trip_index : groupby cumcount
 - 3. Calcul prev_liaison : groupby shift(1)
 - 4. Calcul days_since_prev : difference de dates
 - 5. Calcul user_top_liaison_share : rolling fraction (sans data leakage)
 - 6. Calcul is_self_purchase : AcheteurId == CodeClient
 - 7. Conversion categories -> str pour OrdinalEncoder
 - 8. Selection des 26 colonnes finales

RESULTAT : features.parquet : 491 680 lignes x 26 colonnes



SCRIPT 03 - XGBoost multiclass : 1 011 classes (liaisons)